

System Design Mock #2 — LLM Evaluation & Experimentation Platform

Date: 2026-09-15

Format: Interactive interview (assistant as interviewer, Sid driving)

The Question

Design a production LLM evaluation and experimentation platform — the system that lets 40 product teams run offline evals, A/B test model/prompt versions, and decide what ships. Regulated industry (healthcare/pharma). Ships: fine-tuned model versions, prompt templates, agent configs. Prompts change daily, models weekly. Eval inference spend is the biggest cost line and must grow sublinearly.

Pinned Requirements (from the session)

- **Users:** 40 product teams; each runs evals on every candidate change
- **Artifacts:** versioned prompts, fine-tuned models, agent configs
- **Datasets:** golden sets from hundreds to 10K examples per team
- **Inference:** mix of internal models + external LLM APIs
- **Cadence:** prompts daily, models weekly — a release train, not ad-hoc pushes
- **Metrics:** task accuracy on golden sets + latency + cost/request (standard); teams can define custom metrics (judge-LLM scores, human ratings)
- **Regulated:** full audit trail for every shipped version; quality gates
- **Cost:** eval inference must scale sublinearly

Ideal System Design (how it should be built)

The golden path: from "new prompt" to production

1. **Register.** A team member submits a candidate artifact (prompt template, model version, or agent config bundle) to the artifact registry. The registry hashes the content, assigns an immutable version ID, and stores the blob in S3 with a DynamoDB index entry. Nothing is ever overwritten — new versions only.
2. **Define the experiment.** Through the UI or API, the team creates an experiment: candidate version, baseline version (usually current prod), golden dataset version, metric set (standard + custom), and guardrail thresholds. The feature registry checks

mutual exclusion — no other active experiment on the same feature — and either grants the slot or queues the request.

3. **Execute.** The orchestrator fans out eval jobs. For each (example × candidate, baseline) pair, the inference layer first checks the result cache: key = hash(model version, prompt hash, input hash, decoding params). Cache hit → skip the call. Cache miss → route to batch inference (SageMaker Batch Transform or EC2 Spot fleet for internal models; Bedrock for external APIs). Per-example outputs are written idempotently to S3, keyed by (experiment_id, example_id), with periodic checkpointing so preempted spot jobs resume instead of restarting.
4. **Score.** A scoring job reads raw outputs from S3 and computes metrics: exact-match/accuracy against labels, judge-LLM rubric scores, latency, cost per request. Custom team metrics plug in through a metric interface (a container or Lambda with a standard input/output contract).
5. **Aggregate.** Batch ETL loads per-example results into Redshift. QuickSight dashboards show candidate vs baseline, sliced by input segment, with statistical significance indicators. The team sees results on the next daily refresh (prompts can stream near-real-time for small evals).
6. **Gate.** The quality gate runs automatically: accuracy delta vs baseline must clear the threshold AND all guardrails must pass (toxicity, PII, refusal correctness, latency SLO, cost ceiling). Fail → experiment marked failed, failure recorded for the cross-team aggregation job. Pass → candidate is promotion-eligible.
7. **Release.** Prompts ship through the fast path: config push behind a feature flag, then canary (1 replica → 1–2h bake → 20% traffic/24h → 100%), gating on error/quality metrics with automatic rollback triggers. Models ship through the slow path: staged artifact deployment on the weekly train, heavier gates, human sign-off at each stage (the regulated-industry requirement).
8. **Experiment online (optional).** For product-impact questions offline evals can't answer, the team launches an A/B test: the assignment service sticky-buckets users via hash(user_id), every request logs exposure (user_id, experiment_id → variant), and outcome metrics join in Redshift. Pre-registered sample size; no peeking.
9. **Audit.** Every promotion writes an immutable record: artifact version, dataset version, metric results, gate decisions, approver identities, timestamps — to S3 with Object Lock. This is the evidence package an auditor asks for years later.

Component deep dive

Artifact registry (S3 + DynamoDB). Content-addressed, immutable, versioned. Prompts, model pointers (large weights live in a model store; the registry holds the manifest), agent

config bundles (prompt + tools + orchestration versioned together). DynamoDB holds metadata: version ID, content hash, author, timestamp, lineage (parent version). Alternatives ruled out: a relational DB as primary blob store (wrong tool), mutable versioning (breaks audit).

Feature registry + experiment slots (DynamoDB). Maps feature → owning team → active experiment (at most one). Enforces mutual exclusion so two teams can't mutate the same surface simultaneously. Also the kill-switch registry for flags.

Experiment orchestrator (Step Functions / Airflow). Owns experiment lifecycle state machine: DRAFT → QUEUED → RUNNING → SCORING → GATED → PROMOTED/FAILED. Schedules and monitors eval jobs, handles retries with backoff, enforces per-team concurrency quotas so one team can't starve the fleet.

Inference result cache (ElastiCache Redis, fallback DynamoDB). The cost engine of the platform. Key = SHA256(model_version || prompt_hash || input_hash || temperature || top_p || max_tokens || seed). TTL measured in weeks; eviction by LRU. Only deterministic calls are cached. Expected hit rate 50–70% on overlapping golden sets and nightly re-runs. Without this, inference cost scales linearly with teams × examples; with it, sublinear.

Batch inference fleet (EC2 Spot + SageMaker Batch Transform). Spot for cost (60–90% cheaper), with checkpointing and idempotent per-example writes to bound preemption rework. On-demand fallback for deadline-critical evals. External APIs via Bedrock with per-team rate limits and spend caps.

Scoring workers. Stateless containers. Standard metrics computed in-platform; custom metrics via a plugin contract (Docker image or Lambda, JSON in/out). Judge-LLM scoring runs here too, with judge version pinned and agreement-vs-human tracked as its own metric.

Metrics warehouse (Redshift) + dashboards (QuickSight). Batch ETL from S3 (nightly for models, hourly micro-batch for prompts). Star schema: fact table per (experiment, example) result; dimensions for artifact versions, dataset versions, segments. QuickSight for team dashboards; the failure-aggregation job scans for cross-team patterns.

Operational metrics (CloudWatch). GPU utilization, queue depth, job duration, error rates, cache hit rate, spend-per-team. Alarms on fleet saturation and spend anomalies.

Assignment service (in-house or LaunchDarkly-style). Sticky bucketing via hash(user_id, experiment_id) mod 100 → variant ranges. Deterministic, no per-user state to store. Exposure events streamed to S3 → Redshift.

Quality gate service. Declarative gate config per team: metric thresholds, guardrail list, required approvers. Evaluates automatically on scoring completion; writes the decision + evidence to the audit log.

Release pipelines (two). Prompt pipeline: config push → flag → canary → full. Model pipeline: artifact staging → bake → canary → train promotion. Both with automatic rollback on trigger metrics and manual approval gates where regulation demands.

Audit log (S3 Object Lock, WORM). Append-only. Every state transition that matters — experiment created, gate passed/failed, promotion approved, rollback executed — lands here with actor, timestamp, and artifact hashes. Retention per regulatory schedule (years, not days).

Failure aggregation job (nightly batch over Redshift). Groups failed evals by metric, segment, model, and time window across all teams. Surfaces candidate systemic issues ("12 teams regressed on clinical-note summarization this week") to the platform team.

Data model (key entities)

- **Artifact**(version_id, content_hash, type [prompt|model|agent], blob_uri, parent_version, author, created_at)
- **Dataset**(dataset_id, version, example_count, content_hash, created_at) — immutable versions
- **Experiment**(experiment_id, feature_id, candidate_version, baseline_version, dataset_version, metric_set, status, owner)
- **EvalResult**(experiment_id, example_id, output_uri, metrics_json, cache_hit, computed_at) — idempotent key (experiment_id, example_id)
- **Exposure**(user_id, experiment_id, variant, timestamp)
- **GateDecision**(experiment_id, passed, metric_results, guardrail_results, approver, decided_at)
- **AuditEvent**(event_id, actor, action, artifact_hashes, timestamp) — append-only, WORM

API sketch

- POST /artifacts — register candidate (returns version_id)
- POST /experiments — create experiment (checks feature slot)
- GET /experiments/{id} — status, metrics, gate state
- POST /experiments/{id}/promote — request promotion (runs gate)
- GET /features/{id}/slot — mutual-exclusion check
- POST /assignments — variant for (user_id, experiment_id) [internal]

- Webhooks/event bus for: experiment completed, gate passed/failed, promotion, rollback

Scale math

- 40 teams × 10K examples × (candidate + baseline) = up to 800K inference calls per full eval round. At ~\$0.002–0.01/call (mixed internal/external), a round costs \$1.6K–\$8K before caching; 50–70% cache hit rate cuts that to roughly a third. Daily prompt evals across teams stay in the low thousands of dollars per day — the sublinear story.
- Warehouse: 800K rows/round × ~1KB = under 1GB/day. Trivial for Redshift; the raw S3 outputs dominate (model outputs ~2KB each → ~1.6GB/day, lifecycle-purged per retention policy).
- Assignment service: must handle production request rate with p99 < 5ms — it's a hash computation, stateless, horizontally scalable.

Failure modes

Failure	Mitigation
Spot preemption mid-eval	Checkpoint partial results to S3; idempotent per-example writes; resume from checkpoint
Cache poisoning (stale model outputs)	Model version in cache key; cache bypass for non-deterministic calls
Two teams collide on one feature	Feature registry mutual exclusion; queue/wait
Dataset changed mid-comparison	Immutable dataset versions; experiment pins dataset_version
Peeking inflates significance	Pre-registered sample sizes; sequential testing
Judge-LLM drift	Pin judge version; track human-agreement metric; alert on drift
Bad prod push	Automatic rollback triggers; separate prompt/model pipelines; audit log for forensics
Spend runaway	Per-team spend caps; cache hit-rate alarms; cost attribution dashboards

Session Mistakes & Corrections

10. **S3 as the queryable metrics store.** Initially proposed dashboards querying S3 directly. Corrected to: S3 = raw blob store → cron/batch ETL → Redshift = queryable warehouse → QuickSight dashboards. Lesson: always separate the raw store from the serving/query layer.

11. **Grafana/Prometheus inversion.** Said "Grafana for system perf, Prometheus for other metrics" — backwards. Grafana visualizes; Prometheus (or CloudWatch) stores. Self-corrected to full-AWS: CloudWatch for operational metrics.
12. **"Checkpointing is git."** Git gives reproducibility (re-run the same code), not job checkpointing. True checkpointing = persisting partial per-example results (e.g., to S3) so a preempted spot job resumes at example 7,000 instead of restarting. Different mechanisms, different purposes.
13. **"Eat the rework" on spot preemption.** Acceptable as a conscious tradeoff, but for 10K-example evals a preemption late in the job wastes hours of GPU time and delays iteration. Mitigation: idempotent per-example writes + checkpointing, so rework is bounded.
14. **Root-cause feedback loop was hand-wavy.** Sharpened from "the system shows root cause" to concrete mechanisms: (a) diff candidate outputs against baseline on failed examples, (b) cross-team aggregation of failures by metric/segment across 20+ teams to surface systemic issues. Label aspirational parts as aspirational.
15. **Prompts and models initially shared one rollout pipeline.** Separated after probing: different artifacts, different cadence, different blast radius.
16. **Forgot the term "sticky bucketing"** ("digital twin kind of"). Hash(user_id) → variant, consistent across requests. Without it, users flip variants and metrics become noise.
17. **Didn't cover:** statistical rigor (sample size, significance, the peeking problem), judge-LLM calibration, golden-dataset versioning (dataset drift invalidates comparisons), the audit log implementation, safety/red-team evals for healthcare.

Technical Concepts (interview angles)

18. **Offline eval vs online experiment** — Offline: candidate vs baseline on golden sets, cheap, no user risk, but distribution shift. Online: A/B on live traffic, ground truth, but slow and risky. A real platform needs both; the gate is offline → canary → online.
19. **Golden datasets** — Curated, labeled, versioned eval sets. Versioning matters: changing the dataset invalidates historical comparisons. Treat datasets as immutable artifacts with versions, like code.
20. **Judge-LLM (LLM-as-a-judge)** — Using a strong model to score outputs on rubrics (helpfulness, safety). Cheap and scalable, but needs calibration against human labels; judges have biases (verbosity, position). Always report judge agreement rates.
21. **Content-addressed inference cache** — Key = hash(model version, prompt, input, decoding params). Biggest cost lever in eval platforms. Only valid for deterministic decoding; model version in the key prevents stale hits.

22. **Sticky bucketing** — Deterministic user→variant assignment via hashing. Guarantees consistency; enables clean exposure logging. Alternative: random per-request (noisy, wrong).
23. **Exposure logging** — Every served request logs (user_id, experiment_id, variant). Joined with outcome metrics in the warehouse; without it you can't attribute metric movement to the experiment.
24. **The peeking problem** — Checking experiment results repeatedly and stopping when significant inflates false positives. Fix: pre-registered sample sizes / sequential testing methods.
25. **Canary deployments** — Staged rollout (1 replica → 20% → 100%) watching error/quality metrics with automatic rollback. For LLMs, watch quality regressions, not just latency/errors.
26. **Feature flags / experimentation service** — Decouples deployment from release; the assignment and kill-switch layer. LaunchDarkly-style or in-house.
27. **Mutual exclusion in experimentation** — One active experiment per feature/surface to avoid interaction effects. Managed via a registry with ownership.
28. **Batch vs streaming metrics pipelines** — Batch (nightly ETL to warehouse) is cheaper and fine for slow-moving metrics; streaming needed for canary guardrails. Match freshness to decision latency.
29. **Redshift / data warehouse role** — Queryable, aggregated view over raw event blobs. The dashboard never scans raw S3.
30. **S3 as raw store** — Immutable, cheap, infinite. Right for raw eval outputs and artifacts; wrong as a query layer.
31. **CloudWatch** — Operational metrics: CPU/GPU utilization, job durations, error rates, queue depth. Not the place for business/experiment metrics.
32. **QuickSight** — BI dashboard over the warehouse for PMs/EMs; experiment results, metric trends, failure slices.
33. **Spot instances** — 60–90% cheaper, preemptible. Right for fault-tolerant batch eval; needs checkpointing + idempotent retries to bound rework.
34. **SageMaker Batch Transform** — Managed batch inference; alternative to self-managed spot fleets. Less control, less ops burden.
35. **Idempotency in eval jobs** — Per-example result writes keyed by (experiment_id, example_id) so retries never double-count. Essential with preemptible compute.
36. **Prompt vs model deployment** — Prompts are config (fast push, instant rollback); models are artifacts (staged deploy, bake time). Separate pipelines, separate gates.

37. **Quality gates** — Automated checks before promotion: accuracy $\geq$ baseline + guardrails (toxicity, PII, latency, cost). In regulated industries, gates produce the audit evidence.
38. **Audit trail (regulated)** — Immutable append-only log of: artifact version, eval results, gate decisions, approvers, timestamps. S3 Object Lock or ledger DB. Answers "why did version X ship?" years later.
39. **Baseline diffing** — On failure, show side-by-side candidate vs baseline outputs on failed examples. Cheapest, highest-signal debugging tool.
40. **Cross-team failure aggregation** — Rolling up eval failures across teams by metric/segment surfaces systemic issues (bad data, shared component regression). Platform-level insight individual teams can't see.
41. **Dataset drift** — When production data distribution shifts, golden sets go stale and eval results mislead. Monitor input distribution; refresh golden sets on a schedule.
42. **Human eval / ratings** — Gold standard for quality, expensive and slow. Reserve for gate decisions and judge calibration, not every iteration.
43. **Guardrail metrics** — Safety/quality floors (toxicity, PII leakage, refusal correctness) that can veto a ship regardless of accuracy gains. Especially critical in healthcare.
44. **Cost attribution** — Per-team, per-experiment inference spend tracking. Makes the "sublinear growth" constraint enforceable; drives cache adoption.
45. **Release train** — Fixed cadence (weekly model train, daily prompt window) instead of ad-hoc ships. Batches changes, simplifies rollback, gives auditors a rhythm.
46. **Rollback triggers** — Predefined, automatic: error-rate regression $> X$, guardrail violation, latency SLO breach. Manual: human review at stage gates for regulated sign-off.
47. **Temperature/decoding params in cache keys** — Any param affecting output distribution must be in the key. Sampling (temp > 0) breaks cacheability entirely.
48. **Agent config versioning** — Agents = prompts + tools + orchestration logic; version as a bundle so evals reproduce the full system, not just the model.
49. **Stratified sampling for eval** — Ensure golden sets cover segments (language, intent, edge cases) proportionally; aggregate metrics hide segment regressions.
50. **Segment-level analysis** — Slice results by input segment to find where the candidate regresses even when top-line accuracy improves. The "accuracy up, but worse for segment Y" trap.
51. **Shadow mode** — Run candidate on live traffic without serving its outputs; compares against production reality with zero user risk. Between offline eval and canary.

52. **Champion/challenger** — Persistent pattern: current prod model (champion) vs candidates (challengers) continuously evaluated; promotion when a challenger wins on the full gate.

What the interviewer was really testing (EM lens)

- **Platform thinking:** does the design serve 40 teams, or one team 40 times? (Failure aggregation, shared cache, self-serve.)
- **Cost ownership:** the constraint was explicit; the cache and spot-instance answers were the graded moments.
- **Regulated-industry judgment:** audit trails, gates, sign-offs — not as buzzwords but as concrete components.
- **Separating concerns:** raw vs queryable stores, prompts vs models, offline vs online, batch vs real-time.
- **Intellectual honesty:** labeling the root-cause loop as partly aspirational and sharpening it when challenged.